Практическая работа № 3

Прогнозирование энергопотребления с помощью LSTM и GRU

Дисциплина: «Прикладной искусственный интеллект»

Студентка: Бахорина Ольга Александровна

Группа: R4176c · Курс: 1 курс магистратуры

Основной результат

В работе построен конвейер прогнозирования почасового энергопотребления American Electric Power на один час вперёд. На вход модели подаётся недельная история из 168 часов и шесть циклических календарных признаков. Конфигурация выбиралась по validation loss, а test использовался только для итоговой проверки. Лучший результат показала однослойная GRU с 16 257 параметрами: на 18 195 тестовых окнах MAE составила 112,16 МВт, RMSE — 147,94 МВт, R^2 — 0,9965, sMAPE — 0,7635%. По MAE модель улучшила лучший простой baseline на 73,14%.

1. Цель работы

Освоить полный цикл прогнозирования временного ряда с помощью рекуррентных нейронных сетей PyTorch: проверить и очистить данные, исследовать сезонность, сформировать признаки и скользящие окна без утечки информации, сравнить LSTM и GRU, выбрать конфигурацию по validation и независимо оценить её на test.

2. Задачи работы

1) Загрузить Hourly Energy Consumption и выделить ряд AEP_MW.
2) Проверить типы данных, диапазон дат, пропуски и повторяющиеся временные метки.
3) Восстановить регулярную почасовую шкалу причинным способом.

4) Исследовать динамику, распределение, суточную, недельную и годовую сезонность.
5) Обосновать недельное окно истории по автокорреляции обучающей части ряда.
6) Добавить циклические календарные признаки и выполнить разбиение 70 15 15.
7) Нормализовать признаки только по train и сформировать скользящие окна.
8) Реализовать универсальную архитектуру регрессора на основе LSTM и GRU.
9) Настроить Huber loss, AdamW, gradient clipping, scheduler и EarlyStopping.
10) Сравнить три конфигурации по полной validation выборке.
11) Сопоставить лучшую RNN с persistence и weekly seasonal baseline.
12) Рассчитать MAE, RMSE, R^2 и sMAPE, исследовать остатки и худшие ошибки.
13) Сохранить модель, scaler, метаданные и проверить отдельный прогноз следующего часа.

3. Исходные данные

Использован публичный набор Hourly Energy Consumption с Kaggle. Он объединяет многолетние оценки почасовой нагрузки нескольких регионов энергосистемы PJM. Для эксперимента выбран файл AEP_hourly.csv с временной меткой Datetime и целевой переменной AEP_MW, измеряемой в мегаваттах. Исходный CSV содержит 121 273 строки за период с 1 октября 2004 года по 3 августа 2018 года.

Таблица 1 — Исходные характеристики ряда

Показатель	Значение	Пояснение
Строк в CSV	121 273	Наблюдения до очистки
Уникальных дат	121 269	Четыре временные метки повторяются
Пропуски AEP_MW	0	Пустых значений в исходных строках нет
Минимум	9 581 МВт	Наименьшая исходная нагрузка
Среднее	15 499,51 МВт	Средняя нагрузка исходного ряда
Медиана	15 310 МВт	Центральное значение распределения
Максимум	25 695 МВт	Наибольшая исходная нагрузка
Период	2004-10-01 - 2018-08-03	Почти четырнадцать лет

CSV первоначально расположен не по времени: первые строки относятся к декабрю 2004 года, хотя минимальная дата находится в октябре. Поэтому сортировка является обязательной частью подготовки. Четыре дубля времени могут быть связаны с переходами на зимнее время. Их нельзя оставлять, поскольку скользящее окно должно содержать ровно один объект на каждый час.

4. Ход работы

Подготовка окружения и воспроизводимость

В начале notebook проверяется наличие NumPy, pandas, matplotlib, PyTorch, KaggleHub, joblib, scikit-learn и seaborn. Недостающие пакеты устанавливаются автоматически, после чего импортируются средства для работы с файлами, таблицами, графиками, scaler, метриками, Dataset и DataLoader.

Листинг 1 — Фиксация случайности и выбор устройства

```
SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    torch.use_deterministic_algorithms(True, warn_only=True)

set_seed()
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Один seed задаётся для Python, NumPy, CPU и всех CUDA устройств. Детерминированные алгоритмы уменьшают различия между повторными запусками, а режим warn_only не останавливает выполнение, если отдельная GPU операция не имеет детерминированной реализации. cudnn.benchmark отключён, чтобы библиотека не меняла алгоритм после измерения производительности. Полная побитовая идентичность между разными версиями PyTorch и CUDA всё равно не гарантируется. Текущий запуск выполнен на CUDA.

Таблица 2 — Среда выполнения

Компонент	Версия	Роль
Python	3.13.15	Выполнение ноутбука
PyTorch	2.11.0+cu128	RNN и обучение на GPU
NumPy	2.1.3	Численные преобразования
pandas	2.2.3	Временной индекс и таблицы
scikit-learn	1.6.1	Scaler и метрики регрессии
seaborn	0.13.2	Графики распределений
KaggleHub	1.0.2	Загрузка набора
joblib	1.6.0	Сохранение scaler

Параметры эксперимента

Таблица 3 — Основные параметры

Параметр	Значение	Назначение
TARGET	AEP_MW	Нагрузка AEP в мегаваттах
LOOKBACK	168 часов	История за одну неделю
HORIZON	1 час	Горизонт прогноза
TRAIN_RATIO	0,70	Начальная обучающая часть
VAL_RATIO	0,15	Следующая часть для выбора модели
BATCH_SIZE	256	Число окон в мини пакете
SEARCH_FRACTION	1/3	Доля train для поиска
SEARCH_EPOCHS	6	Бюджет одной пробной конфигурации
MAX_EPOCHS	25	Бюджет финального обучения
NUM_WORKERS	0	Совместимость с Colab и Windows

Результаты эксперимента сохраняются вместе с параметрами модели, таблицами, графиками и объектами масштабирования, чтобы итоговый запуск можно было воспроизвести без повторного подбора конфигурации.

Загрузка и первичная проверка

Листинг 2 — Загрузка и однозначный поиск файла ряда

```
downloaded_dir = Path(kagglehub.dataset_download(DATASET_SLUG))
candidates = list(downloaded_dir.rglob("AEP_hourly.csv"))
if len(candidates) != 1:
   raise FileNotFoundError(...)

raw_df = pd.read_csv(candidates[0])
raw_df["Datetime"] = pd.to_datetime(raw_df["Datetime"], errors="raise")
```

KaggleHub возвращает каталог набора, а rglob ищет файл независимо от структуры вложенных папок. Проверка len(candidates) защищает от двух противоположных ошибок: отсутствия нужного файла и неоднозначного выбора между несколькими копиями. errors='raise' при преобразовании Datetime немедленно выявляет повреждённые даты. После загрузки код выводит форму, первые строки, типы, количество пропусков, число уникальных значений и описательные статистики.

Очистка и регулярная временная шкала

Листинг 3 — Агрегация дублей и причинное заполнение разрывов

```
series_df = (
    raw_df[["Datetime", TARGET]]
    .groupby("Datetime")[TARGET].mean()
    .sort_index().to_frame()
)

full_index = pd.date_range(series_df.index.min(), series_df.index.max(), freq="1h")
series_df = series_df.reindex(full_index)
series_df[TARGET] = series_df[TARGET].ffill()
```

Сначала строки объединяются по Datetime. Если одна временная метка встречается несколько раз, в ряд попадает среднее значение AEP_MW. Затем индекс сортируется и сравнивается с полным почасовым диапазоном. После удаления дублей обнаруживаются 27 отсутствующих часов. reindex создаёт для них строки с NaN, а ffill переносит последнее известное значение вперёд.

В описании notebook встречается слово «интерполяция», однако в коде используется forward fill. Это принципиально для временного ряда: линейная интерполяция может учитывать следующее значение, тогда как ffill использует только уже известное прошлое. Перед заполнением отдельно проверяется первая строка, поскольку причинно восстановить её при пропуске невозможно.

Таблица 4 — Результат очистки

Показатель	Значение	Смысл
Строк после регуляризации	121 296	Полный почасовой индекс
Восстановлено часов	27	Отсутствующие временные метки
Заполнено значений	27	Только предыдущим известным значением
Шаг времени	1 час	Проверен через разность индекса
Конец train	2014-06-08 19:00	Граница фиксируется до EDA
Конец validation	2016-07-05 21:00	Следом начинается test

После очистки assert-проверки подтверждают возрастающий индекс, отсутствие повторов и NaN, а также почасовой шаг между соседними наблюдениями. Границы train и validation фиксируются сразу, поэтому

последующие решения о сезонности, окне и признаках принимаются только по обучающему интервалу.

Разведочный анализ

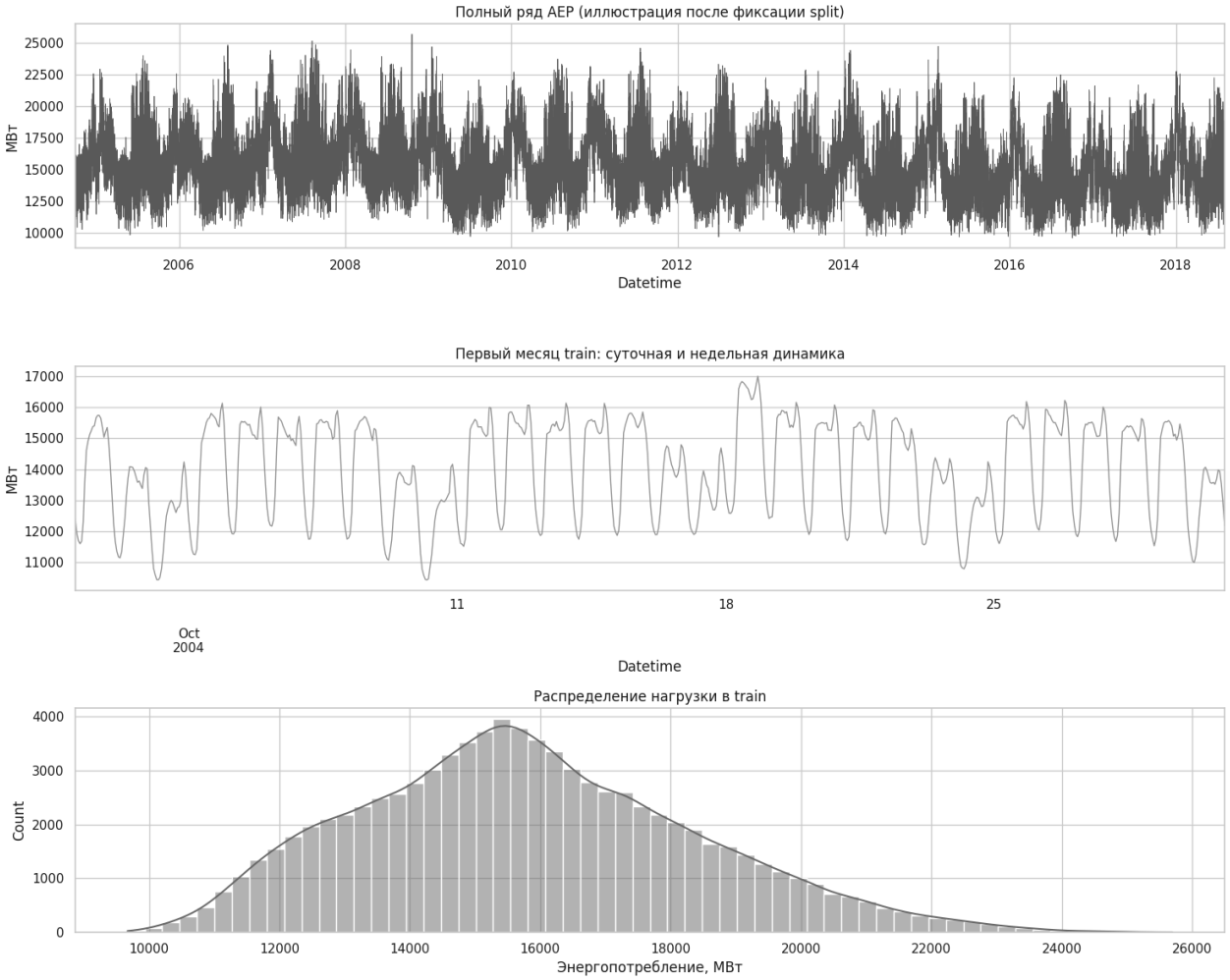


Рисунок 1 — Общая динамика, первый месяц train и распределение нагрузки

Верхний график показывает многолетние изменения амплитуды и уровня нагрузки. Он строится после фиксации границ и используется только как иллюстрация. Средний график увеличивает первый месяц train: внутри каждого дня наблюдается выраженный минимум ночью и рост в дневные часы, а высота пиков меняется по дням недели. Гистограмма имеет широкий диапазон примерно от 9,5 до 25,7 тыс. МВт и правый хвост, поэтому редкие высокие нагрузки способны заметно влиять на квадратичную ошибку.

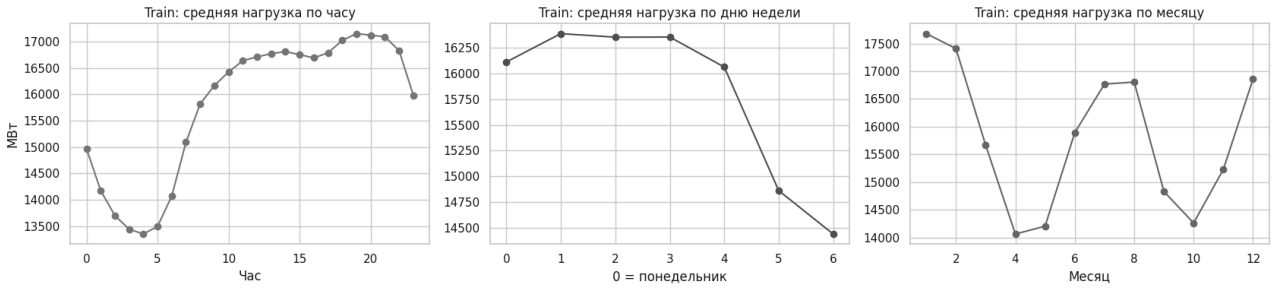


Рисунок 2 — Средняя нагрузка по часу, дню недели и месяцу в train

Почасовой профиль имеет минимум около четырёх часов утра и максимум вечером. Средняя нагрузка наиболее высока в начале рабочей недели и заметно снижается в субботу и воскресенье. Годовой профиль не является простой синусоидой: зимние и летние месяцы выше весенних и осенних, что соответствует отопительной и охладительной нагрузке. Эти зависимости объясняют добавление признаков часа, дня недели и дня года.

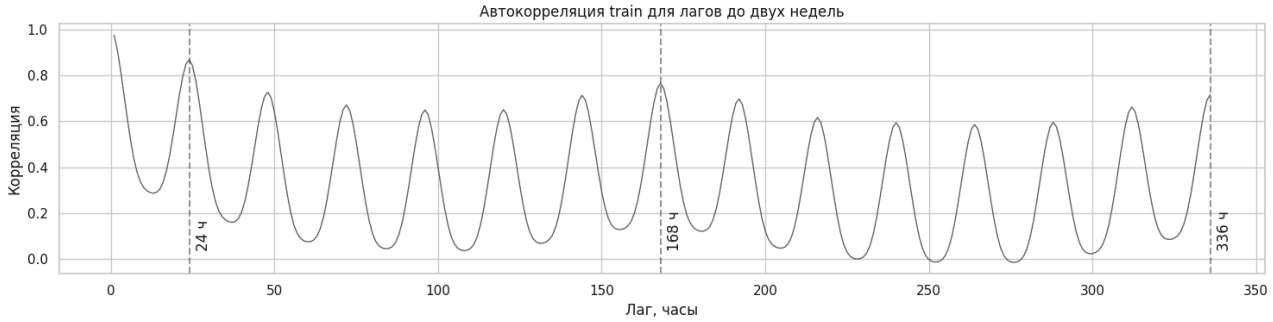


Рисунок 3 — Автокорреляция train на лагах до двух недель

Автокорреляция измеряет сходство ряда с его сдвинутой копией. Для лага 24 часа она равна 0,8685, для 168 часов 0,7659, для 336 часов 0,7159. Повторяющиеся максимумы через каждые сутки и высокая корреляция через неделю подтверждают суточную и недельную сезонность. Поэтому LOOKBACK равен 168: модель видит семь полных суточных циклов и один полный недельный контекст. Значения рассчитаны только по train.

Календарные признаки

Листинг 4 — Циклическое кодирование календаря

```
hour = result.index.hour.to_numpy()
day_of_week = result.index.dayofweek.to_numpy()
day_of_year = result.index.dayofyear.to_numpy()

result["hour_sin"] = np.sin(2 * np.pi * hour / 24)
result["hour_cos"] = np.cos(2 * np.pi * hour / 24)
result["dow_sin"] = np.sin(2 * np.pi * day_of_week / 7)
result["dow_cos"] = np.cos(2 * np.pi * day_of_week / 7)
result["doy_sin"] = np.sin(2 * np.pi * day_of_year / 365.25)
result["doy_cos"] = np.cos(2 * np.pi * day_of_year / 365.25)
```

Обычный номер часа создаёт искусственный разрыв: 23 и 0 численно далеки, хотя соседствуют во времени. Пара синуса и косинуса помещает значения на окружность и сохраняет циклическую близость. То же сделано для семи дней недели и 365,25 дня года. Вход модели содержит семь признаков: нормализованную AEP_MW и шесть календарных координат. Они вычисляются только из известной временной метки, поэтому не содержат будущую нагрузку.

Таблица 5 — Входные признаки модели

Признак	Период	Что сообщает модели
AEP_MW	нет	Предыдущие значения нагрузки
hour_sin и hour_cos	24 часа	Положение внутри суток
dow_sin и dow_cos	7 дней	Положение внутри недели
doy_sin и doy_cos	365,25 дня	Положение внутри года

Хронологическое разбиение

Таблица 6 — Границы выборок

Выборка	Наблюдений	Начало	Конец
Train	84 907	2004-10-01 01:00	2014-06-08 19:00
Validation	18 194	2014-06-08 20:00	2016-07-05 21:00
Test	18 195	2016-07-05 22:00	2018-08-03 00:00

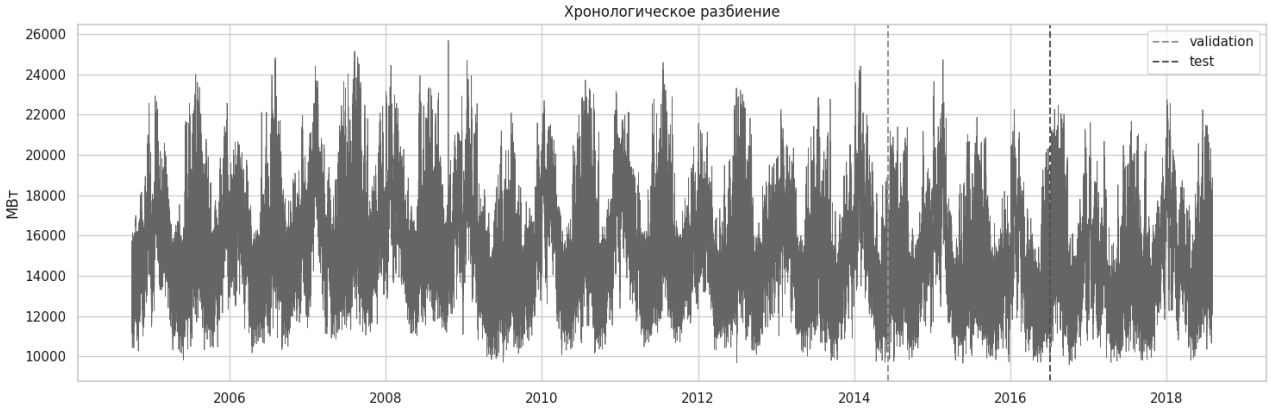


Рисунок 4 — Хронологическое разбиение временного ряда

Перемешивать временной ряд перед разбиением нельзя: случайный split поместил бы будущие режимы нагрузки в обучение и завысил качество. Первые 70% наблюдений образуют train, следующие 15% используются для настройки, последние 15% имитируют будущее и остаются test. Внутри train DataLoader перемешивает уже готовые окна, но содержимое каждого окна и порядок его 168 часов не меняются.

Нормализация без утечки

Листинг 5 — Обучение scaler только на train

```
feature_scaler.fit(feature_df.iloc[:train_end][FEATURES])
target_scaler.fit(feature_df.iloc[:train_end][[TARGET]])

features_scaled = feature_scaler.transform(feature_df[FEATURES]).astype("float32")
target_scaled = target_scaler.transform(feature_df[[TARGET]]).astype("float32").ravel()
```

StandardScaler сохраняет среднее и стандартное отклонение обучающего интервала. После преобразования среднее каждого train признака близко к нулю, а стандартное отклонение к единице. Validation и test преобразуются этими же параметрами. Если fit выполнить на всём ряду, среднее и масштаб будут учитывать будущие значения и создадут утечку. Отдельный target_scaler нужен для обратного перевода прогноза из

нормализованной шкалы в мегаватты. Приведение к float32 соответствует типу весов PyTorch и уменьшает расход памяти.

Скользящие окна

Листинг 6 — Формирование одного обучающего примера

```
target_idx = int(self.target_indices[item])
window_end = target_idx - self.horizon + 1
window_start = window_end - self.lookback
X = self.features[window_start:window_end]
y = self.target[target_idx]
return X, y
```

При HORIZON=1 окно заканчивается непосредственно перед целевым часом. X имеет форму 168 на 7, а y является одним нормализованным числом. Первый train target получает индексы 0-167 как историю. В validation и test разрешён контекст из предыдущей части ряда, поскольку в реальном прогнозе прошлое известно; при этом сама целевая метка всегда принадлежит соответствующему split. Такой подход не теряет первые 168 целей на каждой новой границе и не переносит будущие ответы назад.

Таблица 7 — Число скользящих окон

Выборка	Окон	Особенность
Train	84 739	Первые 168 часов служат начальной историей
Validation	18 194	Цели полностью внутри validation
Test	18 195	Цели полностью внутри test
Search subset	28 246	Случайная фиксированная треть train

Поисковая подвыборка строится случайно с seed 42, индексы затем сортируются. Это лучше систематического выбора каждого третьего часа, который мог бы взаимодействовать с периодичностью ряда. Проверка подтверждает присутствие всех 24 часов суток и семи дней недели. Для честного сравнения каждая конфигурация получает заново созданный DataLoader с тем же генератором случайных чисел. Validation и test не перемешиваются.

Архитектура рекуррентной модели

Листинг 7 — Универсальный регрессор LSTM или GRU

```
rnn_class = {"LSTM": nn.LSTM, "GRU": nn.GRU}[cell]
self.rnn = rnn_class(
    input_size=7, hidden_size=hidden_size,
    num_layers=num_layers, batch_first=True,
    dropout=dropout if num_layers > 1 else 0.0,
)
self.head = nn.Sequential(
    nn.LayerNorm(hidden_size),
    nn.Linear(hidden_size, 32), nn.ReLU(),
    nn.Dropout(dropout), nn.Linear(32, 1),
)

sequence, _ = self.rnn(X)
return self.head(sequence[:, -1]).squeeze(-1)
```

batch_first=True задаёт форму batch на 168 на 7. Рекуррентный слой возвращает скрытое состояние для каждого часа, но регрессионная голова использует только последнее состояние, в котором должна быть собрана информация всего окна. LayerNorm стабилизирует масштаб 64 скрытых признаков. Затем Linear сокращает их до 32, ReLU вводит нелинейность, Dropout уменьшает совместную адаптацию признаков, а последний Linear выдаёт одно число без функции активации, поскольку регрессия не ограничена диапазоном от нуля до единицы.

Таблица 8 — Различия LSTM и GRU

Свойство	**LSTM**	**GRU**
Состояния	Скрытое и отдельная ячейка памяти	Одно объединённое скрытое
Ворота	Забывания, входа и выхода	Сброса и обновления
Параметры	Обычно больше при одинаковом hidden size	Обычно меньше и быстрее
Роль	Гибкое долговременное хранение	Более компактное управление памятью

Параметр dropout внутри nn.LSTM или nn.GRU действует только между несколькими рекуррентными слоями. Поэтому у однослойной победившей GRU recurrent dropout равен нулю, но Dropout 0,15 в полносвязной голове остаётся активным. Это поведение явно закодировано условием num_layers > 1 и предотвращает предупреждение PyTorch.

Функция одной эпохи

Листинг 8 — Общая логика обучения и проверки

```
training = optimizer is not None
model.train(training)

for X_batch, y_batch in loader:
   optimizer.zero_grad(set_to_none=True)  # только train
   with torch.set_grad_enabled(training):
      predictions = model(X_batch)
      loss = criterion(predictions, y_batch)
      if training:
         loss.backward()
         nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
         optimizer.step()
```

Наличие optimizer определяет режим. model.train(True) включает Dropout, а model.train(False) отключает его при validation. torch.set_grad_enabled не строит граф вычислений при проверке и экономит память. zero_grad с set_to_none=True обычно быстрее обнуления тензоров. Норма градиента ограничивается единицей, что особенно важно для рекуррентных сетей, где многократное распространение через время может привести к взрыву градиента. Loss и абсолютная ошибка суммируются с учётом размера batch, поэтому последний неполный пакет получает правильный вес.

Управление процессом обучения

Критерием служит HuberLoss. Для малых ошибок он ведёт себя подобно квадратичной ошибке и обеспечивает гладкий градиент, а для крупных переходит к линейному росту. Это снижает влияние редких скачков нагрузки по сравнению с MSE. AdamW использует адаптивные шаги и отделяет weight decay от обновления градиента. Во всех вариантах weight decay равен 0,0001.

Листинг 9 — Scheduler, checkpoint и ранняя остановка

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=2, min_lr=1e-6
)

improved = val_loss < best_val_loss - 1e-6
if improved:
    best_state = copy.deepcopy(model.state_dict())
    epochs_without_improvement = 0
else:
    epochs_without_improvement += 1

if epochs_without_improvement >= patience:
    break
```

ReduceLROnPlateau отслеживает validation loss и после двух неудачных эпох уменьшает learning rate вдвое, но не ниже 0,000001. Улучшением считается снижение больше одной миллионной, что отсекает численный шум. copy.deepcopy сохраняет независимую копию весов; простая ссылка изменилась бы на следующих эпохах. Для поиска patience равен двум, для финального обучения четырём. В конце всегда восстанавливается лучшее состояние, а не веса последней эпохи.

Подбор гиперпараметров

Таблица 9 — Результаты поиска по validation

Показатель	GRU	LSTM 32	LSTM 64×2
Hidden	64	32	64
Слоёв	1	1	2
Dropout	0,15	0,10	0,20
LR	0,0010	0,0010	0,0005
Лучшая эпоха	5	6	5
Val loss	0,00433	0,00544	0,00980
Val MAE	0,07219	0,07995	0,11112
Время	7,9 с	6,3 с	11,6 с

Все три варианта обучаются на одинаковой трети train и оцениваются на полной validation выборке. Победила однослойная GRU с hidden size 64, dropout 0,15 и learning rate 0,001. Её лучший validation loss 0,00433 достигнут

на пятой эпохе. Однослойная LSTM с 32 скрытыми единицами имеет всего 6 401 параметр и немного уступает. Двухслойная LSTM содержит 54 209 параметров, обучается дольше и показывает худший результат. Это не доказывает общее превосходство GRU, но определяет лучший вариант среди проверенных конфигураций при данном бюджете.

Финальное обучение GRU

После выбора конфигурации модель создаётся заново и получает все 84 739 обучающих окон. Финальный запуск длится 25 эпох и занимает 86,8 секунды на указанном GPU. Минимальный validation Huber loss 0,00162 получен на эпохе 22. До лимита остаются три эпохи без улучшения, поэтому patience 4 не успевает остановить цикл, однако checkpoint двадцать второй эпохи восстанавливается автоматически.

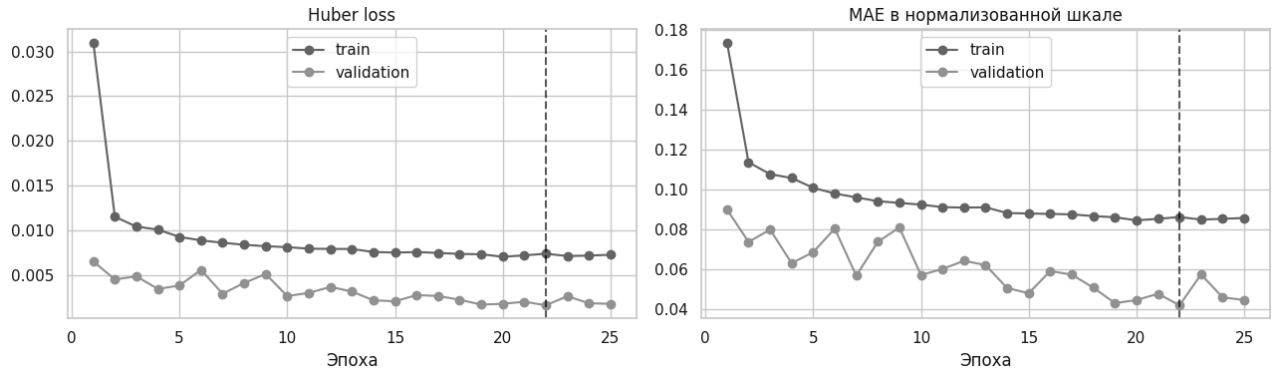


Рисунок 5 — История финального обучения GRU

Train loss быстро падает в первые две эпохи и затем плавно уменьшается. Validation loss колеблется, но достигает новых минимумов на протяжении обучения. Validation ошибка ниже train во многих эпохах. Это возможно, потому что Dropout активен только при обучении, а validation оценивается без случайного отключения признаков. Кроме того, обучающий интервал длиннее и может содержать более разнообразные режимы. График не показывает устойчивого расхождения, характерного для сильного переобучения.

Метрики и простые ориентиры

После обратного преобразования в мегаватты рассчитываются четыре метрики. MAE усредняет абсолютную величину ошибки и легко интерпретируется в МВт. RMSE сильнее штрафует крупные промахи. R^2 показывает долю вариации, объяснённую прогнозом: единица соответствует идеальному совпадению. sMAPE делит абсолютную ошибку на средний масштаб факта и прогноза и выражает результат в процентах; малая добавка epsilon защищает от деления на ноль.

Persistence baseline принимает следующий час равным последнему известному значению. Weekly seasonal baseline копирует нагрузку ровно недельной давности. Первый baseline проверяет, превосходит ли сеть инерцию короткого горизонта. Второй проверяет пользу относительно сильной недельной сезонности. Ни один baseline не обучается и не использует будущие значения.

Таблица 10 — Итоговые результаты на test

Модель	MAE, МВт	RMSE, МВт	R^2	sMAPE, %
Persistence baseline	417,5635	535,0093	0,9537	2,8549
Weekly seasonal baseline	1 444,9626	1 909,7961	0,4100	9,4644
GRU RNN	112,1614	147,9382	0,9965	0,7635

Лучшим простым ориентиром оказался persistence с MAE 417,56 МВт. GRU снизила MAE до 112,16 МВт, то есть на 73,14%. Снижение RMSE до 147,94 МВт показывает, что сеть уменьшила и типичные, и крупные ошибки. R^2 0,9965 означает, что прогноз повторяет почти всю вариативность тестового ряда. Недельный baseline заметно слабее, потому что одинаковый час разных недель может сильно различаться из-за погоды, сезона и изменения уровня нагрузки.



Рисунок 6 — Факт, прогноз GRU и недельный baseline на первых двух неделях test

Линия GRU почти совпадает с фактом и воспроизводит как ночные минимумы, так и дневные пики. Недельный baseline сохраняет общую периодичность, но часто ошибается в амплитуде и уровне. Особенно хорошо видно, что нейросеть быстрее адаптируется к смене режима между соседними днями, поскольку использует всю недавнюю неделю и календарные координаты, а не одно значение недельной давности.

Анализ остатков и крупных ошибок

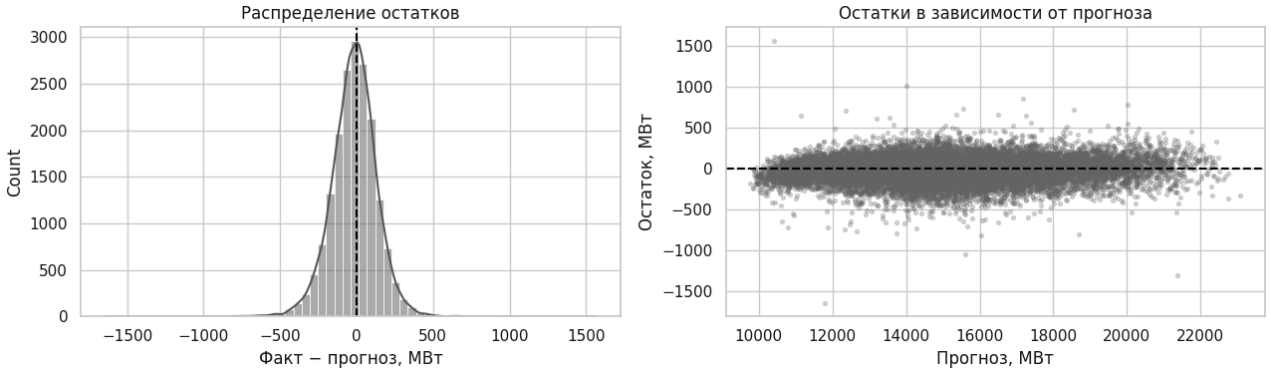


Рисунок 7 — Распределение остатков и их зависимость от прогноза

Остаток определён как факт минус прогноз. Его среднее равно -17,16 МВт, поэтому модель в среднем слегка завышает нагрузку примерно на 17 МВт. Это мало относительно среднего уровня около 15,5 тыс. МВт. Стандартное отклонение остатков равно 146,94 МВт. Гистограмма сосредоточена около нуля, но имеет редкие длинные хвосты. Диаграмма рассеяния не показывает сильной линейной зависимости ошибки от прогнозируемого уровня, хотя на экстремумах разброс становится заметнее.

Таблица 11 — Десять крупнейших абсолютных ошибок

Дата и время	Факт	GRU	Weekly	Остаток	Абс. ошибка
2018-04-24 01:00	10 158	11 799,62	14 706	-1 641,62	1 641,62
2018-04-25 01:00	11 979	10 415,24	13 723	1 563,75	1 563,75
2017-08-21 15:00	20 089	21 383,32	18 215	-1 294,33	1 294,33
2018-04-18 11:00	14 558	15 604,26	15 058	-1 046,25	1 046,25
2018-04-18 12:00	15 031	14 022,86	14 560	1 008,14	1 008,14
2017-03-14 08:00	18 014	17 167,36	14 977	846,64	846,64
2017-02-28 20:00	15 218	16 027,13	14 624	-809,13	809,13
2017-07-07 15:00	17 887	18 690,87	19 268	-803,87	803,87
2016-09-10 02:00	12 959	13 745,72	11 046	-786,72	786,72
2017-08-21 17:00	20 785	20 007,46	18 620	777,54	777,54

Максимальная абсолютная ошибка 1 641,62 МВт возникла ночью 24 апреля 2018 года: фактическая нагрузка резко оказалась ниже прогноза. Уже через сутки знак ошибки сменился и модель недооценила нагрузку на 1 563,75 МВт. Пара противоположных ночных промахов указывает на быстрые изменения режима, которые трудно вывести только из прошлой нагрузки и календаря. Несколько крупных ошибок 21 августа 2017 года могут быть связаны с внешними событиями или погодой, но без дополнительных данных причинную интерпретацию делать нельзя.

Сохранение и восстановление модели

Вместе с весами модели сохраняются параметры конфигурации, порядок входных признаков, горизонт прогноза и параметры масштабирования. При восстановлении модели эти параметры проверяются на согласованность, чтобы исключить применение другой схемы предобработки.

Листинг 10 — Отдельный прогноз следующего часа

```
recent_window = recent_series.iloc[-LOOKBACK:].copy()
if not recent_window.index.to_series().diff().dropna().eq(pd.Timedelta(hours=1)).all():
    raise ValueError("История должна иметь регулярный шаг 1 час")

prepared = add_calendar_features(recent_window)
X = feature_scaler.transform(prepared[feature_names]).astype("float32")
with torch.no_grad():
    prediction_scaled = model(torch.from_numpy(X).unsqueeze(0).to(DEVICE))
prediction = target_scaler.inverse_transform(...).ravel()[0]
```

Функция inference требует не менее 168 наблюдений и проверяет почасовую регулярность. Она повторяет те же календарные признаки, порядок столбцов и scaler, что использованы при обучении. unsqueeze добавляет размер batch, после чего результат возвращается в мегаватты. Для момента 3 августа 2018 года 01:00 модель спрогнозировала 13 695,7 МВт.



Рисунок 8 — Последние три дня ряда и прогноз следующего часа

5. Методические нюансы

Причинность подготовки. Дубли агрегируются внутри одной временной метки, а отсутствующие часы заполняются только последним известным значением.

Разделение по времени. Train, validation и test следуют друг за другом; test не влияет на выбор окна, признаков, scaler, конфигурации или эпохи.

Контекст на границах. Для первых целей validation и test используются известные часы из предыдущего интервала, но сами целевые значения остаются внутри соответствующего split.

Сопоставимость поиска. Все конфигурации получают одинаковый seed, одну и ту же фиксированную треть train и полную validation-выборку.

Критерий выбора. Победитель определяется по минимальному validation Huber loss; тестовые метрики рассчитываются только после завершения выбора.

Время выполнения. Значения 7,9 и 86,8 секунды относятся к конкретному запуску на CUDA и зависят от оборудования и программного окружения.

Ограничение данных. Модель использует только историю нагрузки и календарные признаки; погода, праздники, тарифы и специальные события в признаки не входят.

Ограничение эксперимента. Проверены три конфигурации и один seed, поэтому устойчивость ранжирования LSTM и GRU требует повторных запусков.

6. Вывод

В работе реализован полный цикл прогноза почасового энергопотребления на один час вперёд: от очистки и анализа временного ряда до обучения рекуррентных моделей, выбора конфигурации по validation и независимой оценки на test.

После агрегации четырёх повторяющихся временных меток и причинного заполнения 27 отсутствующих часов получен регулярный ряд из 121 296 наблюдений. На train выражены суточная и недельная сезонности: автокорреляция на лагах 24 и 168 часов составила 0,8685 и 0,7659. Поэтому в модель подаётся недельное окно истории и шесть циклических календарных признаков.

Из трёх проверенных вариантов минимальный validation loss получила GRU с 64 скрытыми единицами и одним слоем. После обучения на полной train-выборке лучший checkpoint был сохранён на эпохе 22 с validation loss 0,00162. На 18 195 тестовых окнах модель достигла MAE 112,16 МВт, RMSE 147,94 МВт, R^2 0,9965 и sMAPE 0,7635%. По сравнению с persistence baseline

MAE снизилась на 73,14%. В этом эксперименте GRU эффективнее использовала недельный контекст, чем простое копирование последнего или недельного значения.

Средний остаток -17,16 МВт указывает на небольшое систематическое завышение прогноза; редкие ошибки до 1 641,62 МВт связаны с резкими изменениями режима. Для дальнейшей проверки модели полезно добавить температуру, праздники и другие внешние признаки, расширить набор гиперпараметров, выполнить несколько запусков с разными seed и рассмотреть многогоризонтный прогноз.

7. Источники и программное обеспечение

Данные

Использован публичный набор Hourly Energy Consumption с Kaggle. Для эксперимента выбран файл AEP_hourly.csv с временной меткой Datetime и целевой переменной AEP_MW, измеряемой в мегаваттах.

Программное обеспечение

Компонент	Версия / назначение
Python	3.13.15
PyTorch	2.11.0+cu128 — рекуррентные модели и обучение на GPU
NumPy	2.1.3 — численные преобразования
pandas	2.2.3 — временной индекс и таблицы
scikit-learn	1.6.1 — StandardScaler и метрики регрессии
seaborn	0.13.2 — графики распределений
KaggleHub	1.0.2 — загрузка набора данных
joblib	1.6.0 — сохранение scaler

Воспроизводимость

Для Python, NumPy, CPU и CUDA используется единый seed=42. Для воспроизводимости вместе с моделью сохраняются параметры конфигурации, порядок признаков и объекты масштабирования.